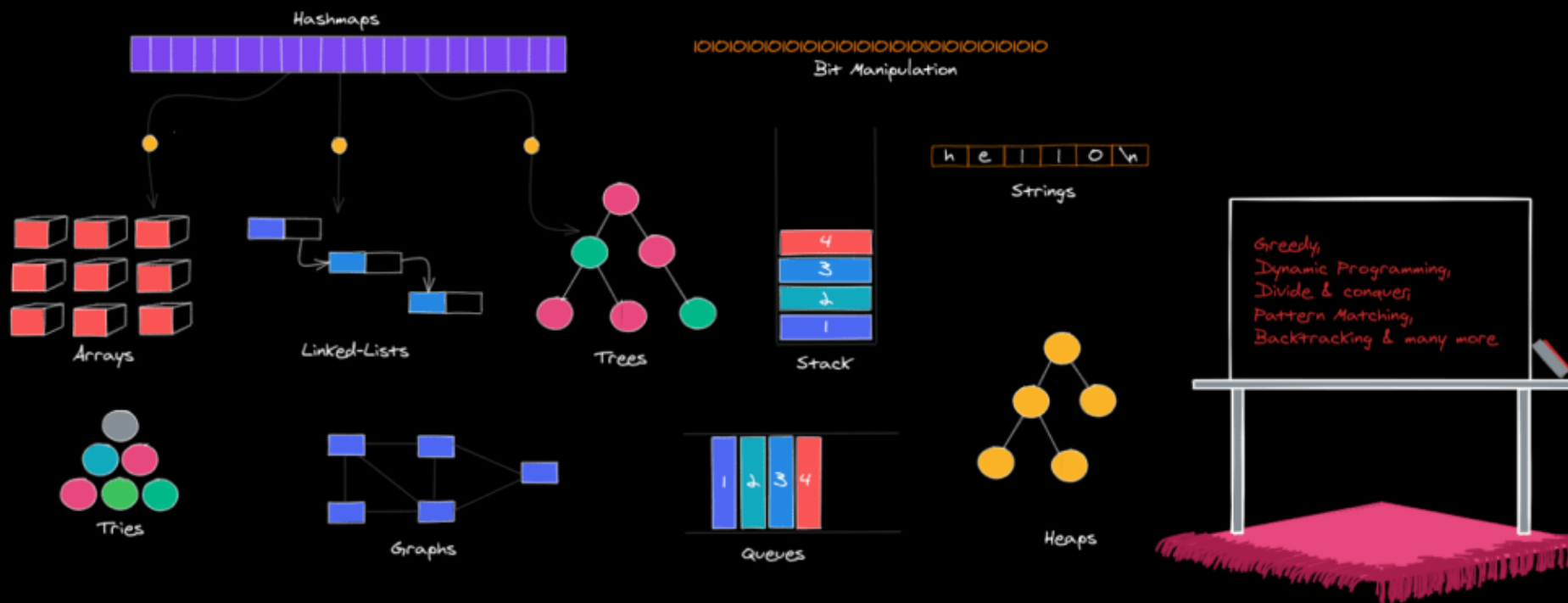




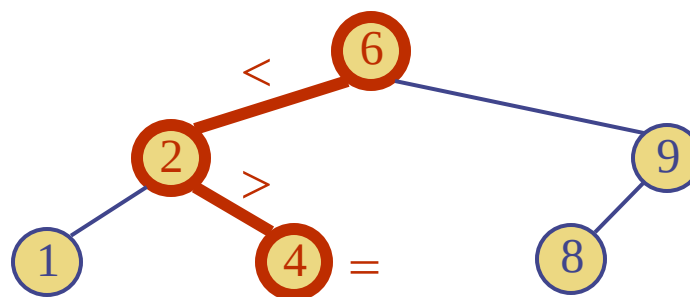
数据结构与算法导论

刘子博

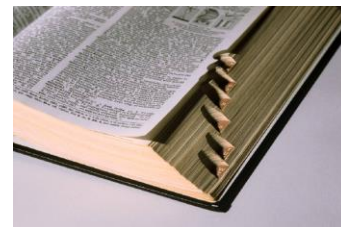
FDSM

liuzibo@fudan.edu.cn

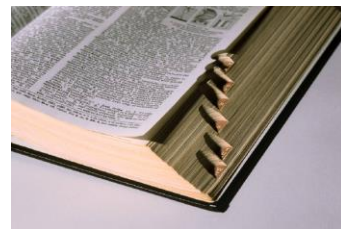
二叉搜索树



有序映射的ADT



- 在有序映射中，键按照一定的顺序进行排列
- 有序映射是映射的一个拓展，除映射支持的行为外，有序映射还支持以下方法：
 - `M.find_min()`: 返回最小键对应的键值对（如映射为空则为None）
 - `M.find_max()`: 返回最大键对应的键值对（如映射为空则为None）
 - `M.find_lt(k)`: 返回小于k的最大键对应的键值对（如不存在则为None）
 - `M.find_le(k)`: 返回小于等于k的最大键对应的键值对（如不存在则为None）



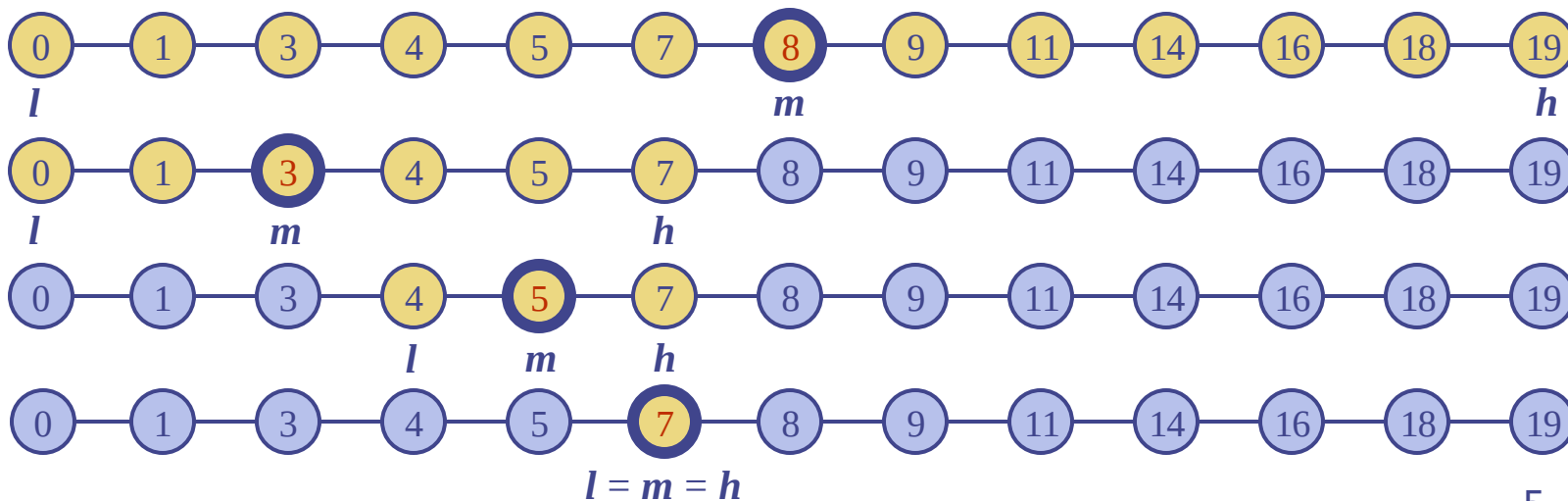
有序映射的ADT

- 在**有序映射**中，键按照一定的顺序进行排列
- 有序映射是映射的一个拓展，除映射支持的行为外，有序映射还支持以下方法：
 - `M.find_gt(k)`: 返回大于k的最小键对应的键值对（如不存在则为None）
 - `M.find_ge(k)`: 返回大于等于k的最小键对应的键值对（如不存在则为None）
 - `M.find-range(start, stop)`: 顺序迭代不小于start且小于stop的所有键值对。如start为None则从最小键开始，如stop为None则迭代至最大键
 - `iter(M)`: 从小到大迭代M的所有键
 - `reversed(M)`: 逆序遍历M的所有键



二分查找

- 在使用数组按键的顺序存储的有序映射上，可以使用二分查找 (binary search) 实现键的快速查找
 - 每次查找将搜索范围减小一半
 - 二分查找在 $O(\log n)$ 时间内完成
- 例: find(7)



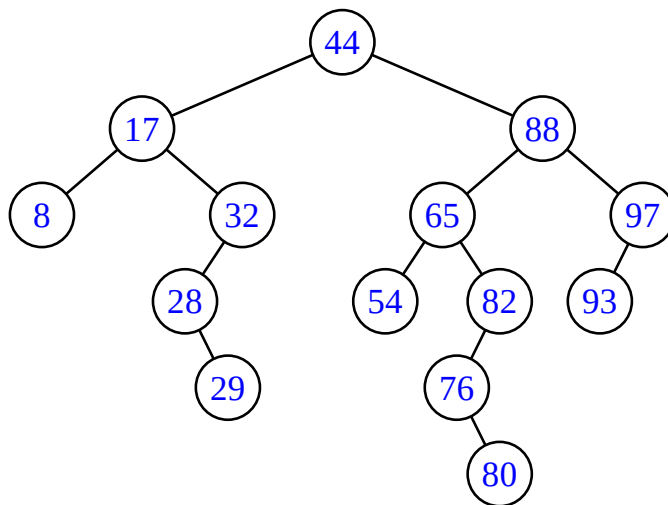
排序检索表



- 排序检索表即使用有序线性列表对有序映射的一种实现
 - 使用基于数组的按键排序的序列
- 性能分析：
 - **查找**使用二分查找，在 $O(\log n)$ 时间内完成
 - **插入**新元素平均需要移动 $n/2$ 个元素，故在 $O(n)$ 时间内完成
 - **删除**新元素平均也需要移动 $n/2$ 个元素，在 $O(n)$ 时间内完成
- 排序检索表仅在元素较少的或最常见的操作为查找（而非插入、删除）的有序映射上较有效率（如信用卡授权）
- 思考：是否有更合适的数据结构？

二叉搜索树

- 二叉搜索树 (binary search tree) 是每个节点 p 均存储一个键 k (或键值对 (k, v)) 的二叉树, 使得:
 - p 的左子树上的键均小于 k
 - p 的右子树上的键均大于 k
- 二叉搜索树的中序遍历恰好返回其存储的有序序列



二叉搜索树的ADT

- 除二叉树ADT中提供的方法外，二叉搜索树的ADT还支持以下方法：
 - first(): 返回存储最小键的节点，如树为空则返回None
 - last(): 返回存储最大键的节点，如树为空则返回None
 - before(p): 返回比节点p的键小的所有节点中最大键所在的节点（即中序遍历中在p之前最后一个被访问的节点），如p是第一个节点则返回None
 - after(p): 返回比节点p的键大的所有节点中最小键所在的节点（即中序遍历中在p之后第一个被访问的节点），如p是最后一个节点则返回None

计算后继节点的位置

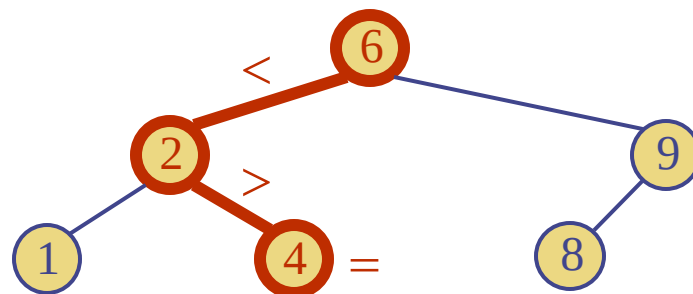
- after(p)方法的算法如下:

Algorithm after(p):

```
if right(p) is not None then {successor is leftmost node in p's right subtree}
    walk = right(p)
    while left(walk) is not None do
        walk = left(walk)
    return walk
else {successor is nearest ancestor having p in its left subtree}
    walk = p
    ancestor = parent(walk)
    while ancestor is not None and walk == right(ancestor) do
        walk = ancestor
        ancestor = parent(walk)
    return ancestor
```

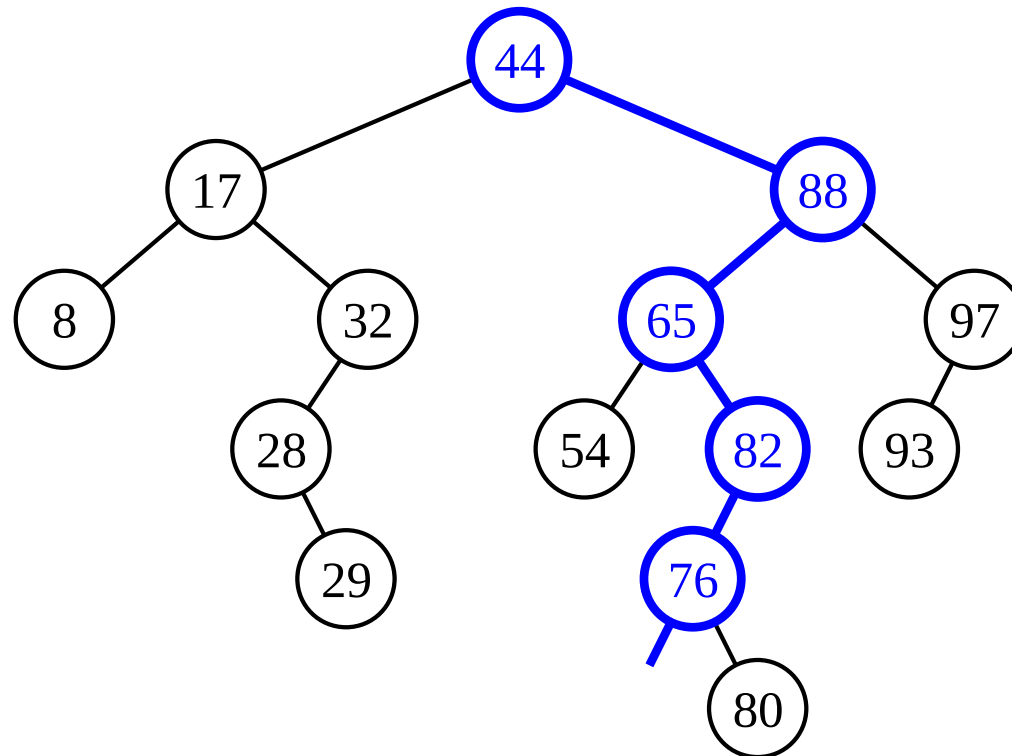
搜索

- 搜索键 k 时，我们从树根开始向下搜索
- 在每个节点 p 上，下一步操作由当前节点的键与 k 的比较结果而决定
 - 如 $p.key()$ 大于 k ，则继续搜索左子树
 - 如 $p.key()$ 等于 k ，则搜索成功且终止
 - 如 $p.key()$ 小于 k ，则继续搜索右子树
- 如最后探查的空子树，则搜索失败
- 例：find(4)



搜索

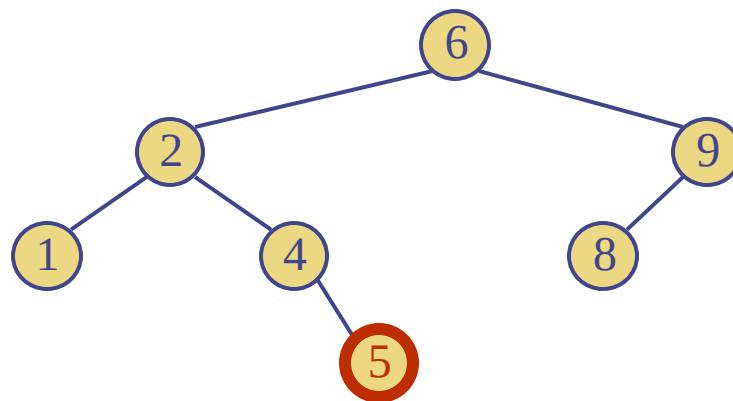
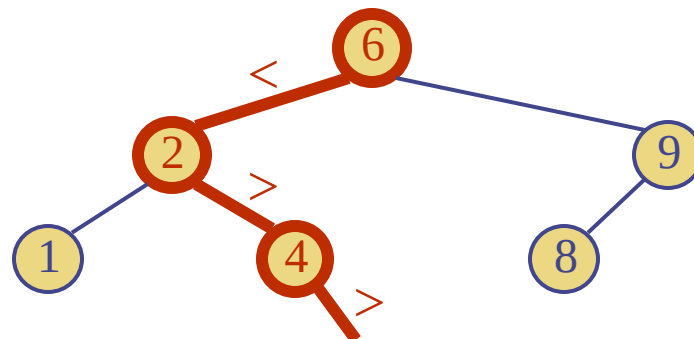
- 例: find(68)



- 12

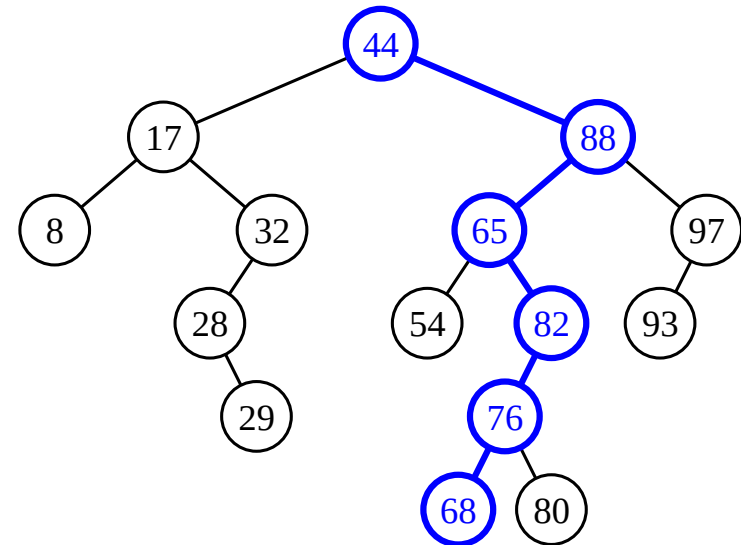
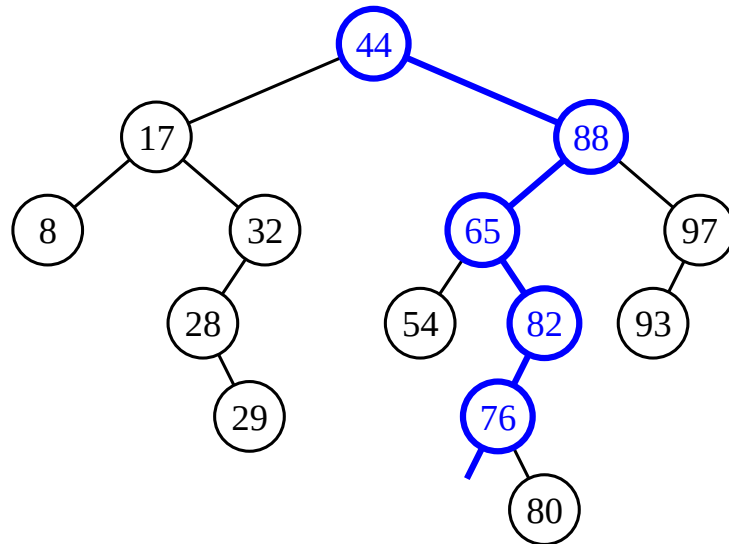
插入

- 对于插入操作，首先搜索键 k 。如找到则重新对其值进行赋值，否则返回搜索失败时最后一个探查的节点 p
 - 如 k 小于 $p.key()$ ，则将包含 k 的新节点作为 p 的左孩子
 - 如 k 大于 $p.key()$ ，则将包含 k 的新节点作为 p 的右孩子
- 例：insert(5)



插入

- 例: insert(68)



插入

- 二叉搜索树的插入算法:

Algorithm **TreeInsert**(T, k, v):

Input: A search key k to be associated with value v

$p = \text{TreeSearch}(T, T.\text{root}(), k)$

if $k == p.\text{key}()$ **then**

 Set p 's value to v

else if $k < p.\text{key}()$ **then**

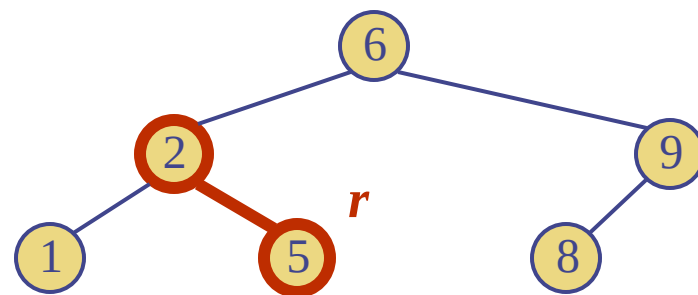
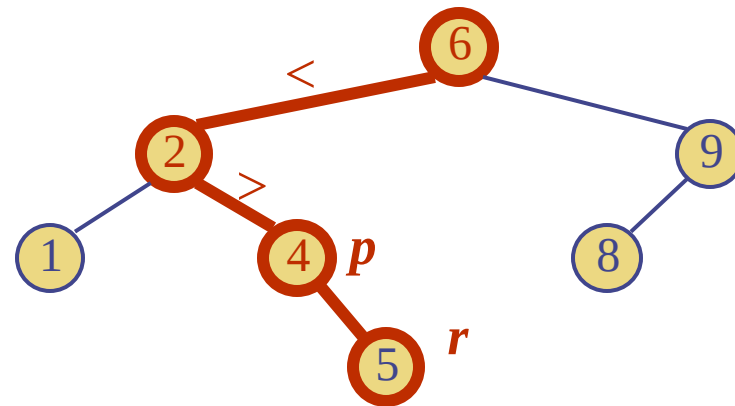
 add node with item (k, v) as left child of p

else

 add node with item (k, v) as right child of p

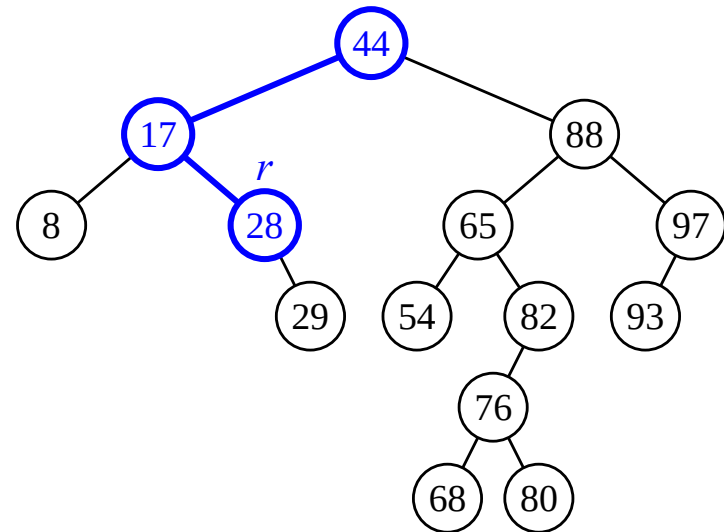
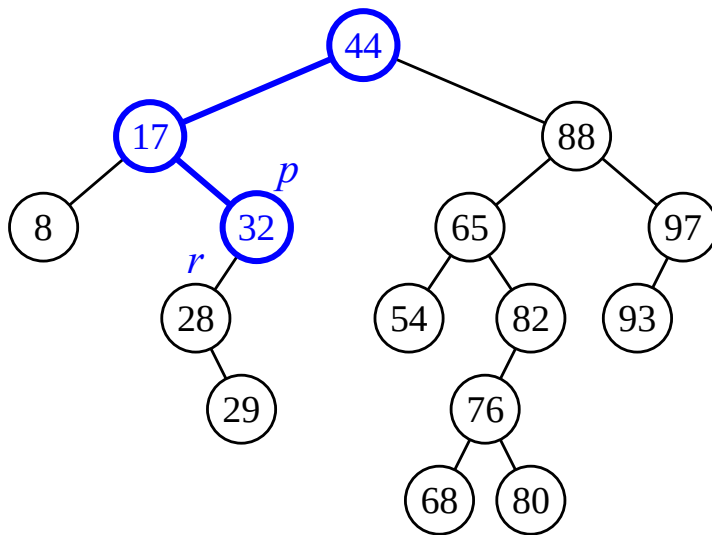
删除

- 删除操作比插入更复杂，因为删除的可能是度为2的节点
- 首先搜索键 k 。如找到，设找到的节点为 p ，则：
 - 如 p 无孩子节点，直接删除即可
 - 如 p 有一个孩子节点 r ，则删除 p 且用其孩子节点替代它
- 例：remove(4)



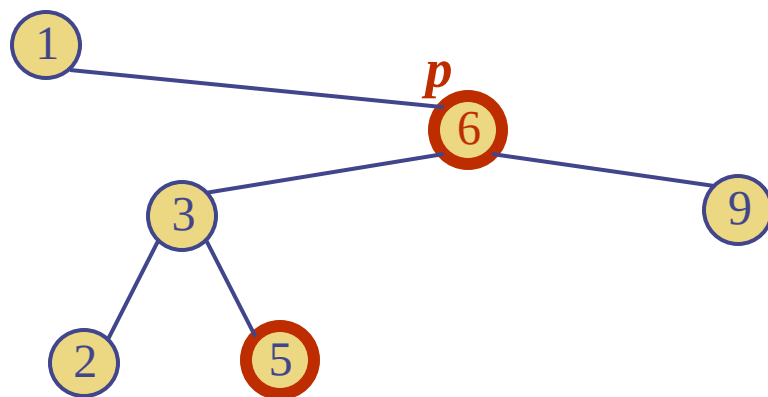
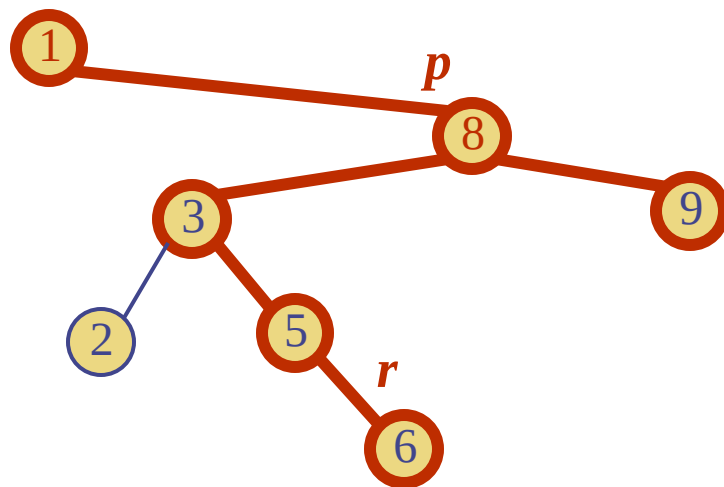
删除

- 例: remove(32)



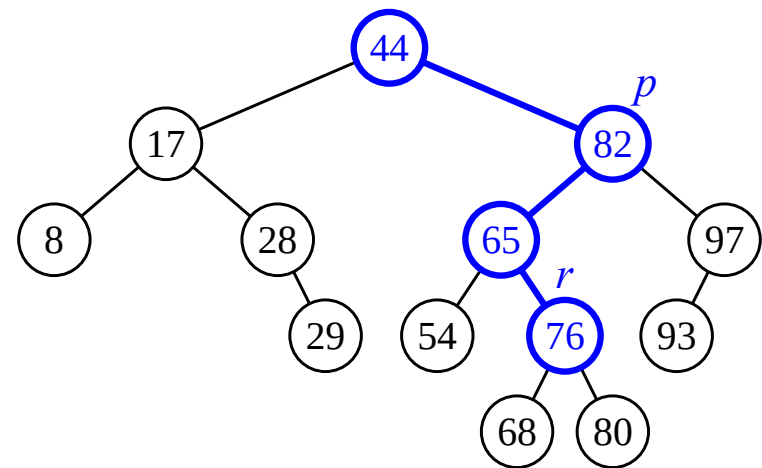
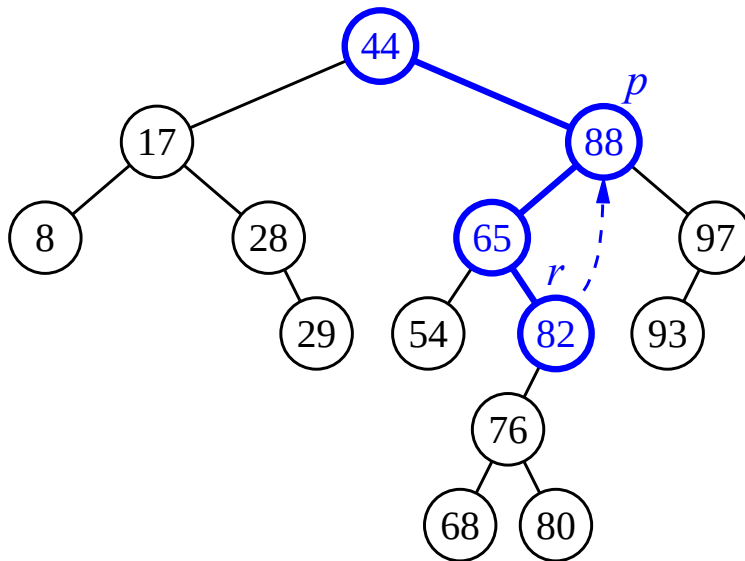
删除

- 如 p 有两个孩子节点
 - 使用 $\text{before}(p)$ 找到中序遍历中节点 p 的前一个节点 r
 - 用节点 r 替代节点 p
 - 删除节点 r
- 此方法可保证二叉搜索树的属性不被破坏
- 例: $\text{remove}(8)$



删除

- 例: remove(88)



二叉搜索树的实现

```
class TreeMap(LinkedBinaryTree, MapBase):
    """Sorted map implementation using a binary search tree."""

    #----- override Node class -----
    class Node(LinkedBinaryTree.Node):
        def key(self):
            """Return key of map's key-value pair."""
            return self.element()._key

        def value(self):
            """Return value of map's key-value pair."""
            return self.element()._value

    #----- nonpublic utilities -----
    def _subtree_search(self, p, k):
        """Return Node of p's subtree having key k, or last node searched."""
        if k == p.key():
            # found match
            return p
        elif k < p.key():
            # search left subtree
            if self.left(p) is not None:
                return self._subtree_search(self.left(p), k)
        else:
            # search right subtree
            if self.right(p) is not None:
                return self._subtree_search(self.right(p), k)
        return p
        # unsuccessful search
```

二叉搜索树的实现

```
def _subtree_first_node(self, p):
    """Return Node of first item in subtree rooted at p."""
    walk = p
    while self.left(walk) is not None:          # keep walking left
        walk = self.left(walk)
    return walk

def _subtree_last_node(self, p):
    """Return Node of last item in subtree rooted at p."""
    walk = p
    while self.right(walk) is not None:         # keep walking right
        walk = self.right(walk)
    return walk

#----- public methods providing "nodal" support -----
def first(self):
    """Return the first Node in the tree (or None if empty)."""
    return self._subtree_first_node(self.root()) if len(self) > 0 else None

def last(self):
    """Return the last Node in the tree (or None if empty)."""
    return self._subtree_last_node(self.root()) if len(self) > 0 else None
```

二叉搜索树的实现

```
def before(self, p):  
    """Return the Node just before p in the natural order.  
  
    Return None if p is the first node.  
    """  
    if self.left(p):  
        return self._subtree_last_node(self.left(p))  
    else:  
        # walk upward  
        walk = p  
        above = self.parent(walk)  
        while above is not None and walk == self.left(above):  
            walk = above  
            above = self.parent(walk)  
        return above
```

二叉搜索树的实现

```
def after(self, p):  
    """Return the Node just after p in the natural order.  
  
    Return None if p is the last node.  
    """  
    if self.right(p):  
        return self._subtree_first_node(self.right(p))  
    else:  
        walk = p  
        above = self.parent(walk)  
        while above is not None and walk == self.right(above):  
            walk = above  
            above = self.parent(walk)  
        return above
```

二叉搜索树的实现

```
def find_node(self, k):
    """Return node with key k, or else neighbor (or None if empty)."""
    if self.is_empty():
        return None
    else:
        p = self._subtree_search(self.root(), k)
        self._rebalance_access(p) # hook for balanced tree subclasses
        return p

def delete(self, p):
    """Remove the item at given Node."""
    if self.left(p) and self.right(p): # p has two children
        replacement = self._subtree_last_node(self.left(p))
        self._replace(p, replacement.element()) # from LinkedBinaryTree
        p = replacement
    # now p has at most one child
    parent = self.parent(p)
    self._delete(p) # inherited from LinkedBinaryTree
    self._rebalance_delete(parent) # if root deleted, parent is None
```


二叉搜索树的实现

```
#----- public methods for (standard) map interface -----
def __setitem__(self, k, v):
    """Assign value v to key k, overwriting existing value if present."""
    if self.is_empty():
        leaf = self._add_root(self._Item(k,v)) # from LinkedBinaryTree
    else:
        p = self._subtree_search(self.root(), k)
        if p.key() == k:
            p.element()._value = v                # replace existing item's value
            self._rebalance_access(p)              # hook for balanced tree subclasses
            return
        else:
            item = self._Item(k,v)
            if p.key() < k:
                leaf = self._add_right(p, item)    # inherited from LinkedBinaryTree
            else:
                leaf = self._add_left(p, item)     # inherited from LinkedBinaryTree
    self._rebalance_insert(leaf)                  # hook for balanced tree subclasses
```

二叉搜索树的实现

```
def __getitem__(self, k):
    """Return value associated with key k (raise KeyError if not found)."""
    if self.is_empty():
        raise KeyError('Key Error: ' + repr(k))
    else:
        p = self._subtree_search(self.root(), k)
        self._rebalance_access(p) # hook for balanced tree subclasses
        if k != p.key():
            raise KeyError('Key Error: ' + repr(k))
        return p.value()

def __delitem__(self, k):
    """Remove item associated with key k (raise KeyError if not found)."""
    if not self.is_empty():
        p = self._subtree_search(self.root(), k)
        if k == p.key():
            self.delete(p)
            return # successful deletion complete
        self._rebalance_access(p) # hook for balanced tree subclasses
    raise KeyError('Key Error: ' + repr(k))
```

二叉搜索树的实现

```
def __iter__(self):
    """Generate an iteration of all keys in the map in order."""
    p = self.first()
    while p is not None:
        yield p.key()
        p = self.after(p)

def find_min(self):
    """Return (key,value) pair with minimum key (or None if empty)."""
    if self.is_empty():
        return None
    else:
        p = self.first()
        return (p.key(), p.value())

def find_ge(self, k):
    """Return (key,value) pair with least key greater than or equal to k.

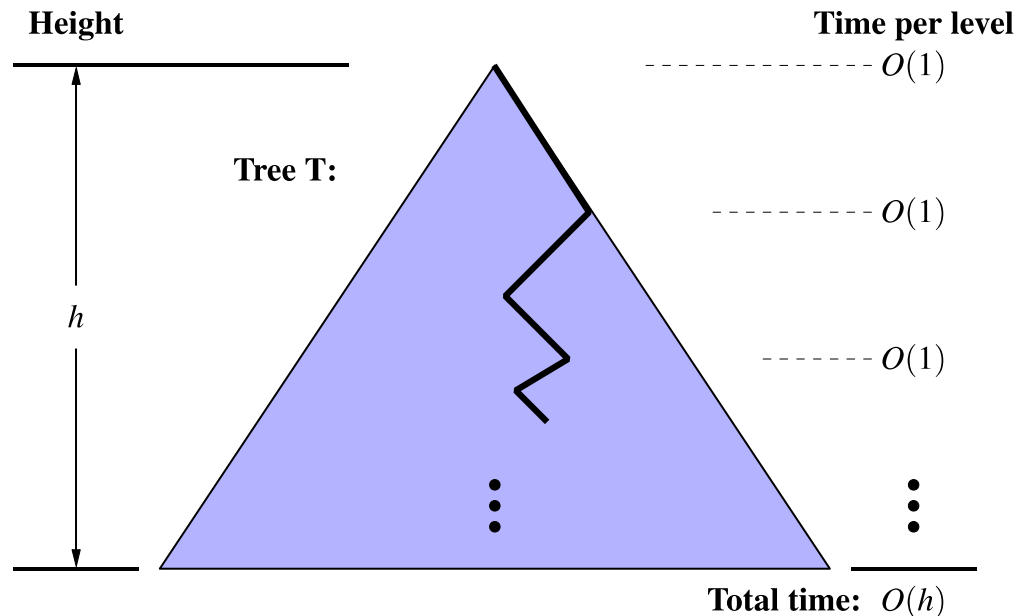
    Return None if there does not exist such a key.
    """
    if self.is_empty():
        return None
    else:
        p = self.find_node(k)                                # may not find exact match
        if p.key() < k:                                       # p's key is too small
            p = self.after(p)
        return (p.key(), p.value()) if p is not None else None
```

二叉搜索树的实现

```
def find_range(self, start, stop):  
    """Iterate all (key,value) pairs such that start <= key < stop.  
  
    If start is None, iteration begins with minimum key of map.  
    If stop is None, iteration continues through the maximum key of map.  
    """  
    if not self.is_empty():  
        if start is None:  
            p = self.first()  
        else:  
            # we initialize p with logic similar to find_ge  
            p = self.find_node(start)  
            if p.key() < start:  
                p = self.after(p)  
        while p is not None and (stop is None or p.key() < stop):  
            yield (p.key(), p.value())  
            p = self.after(p)
```

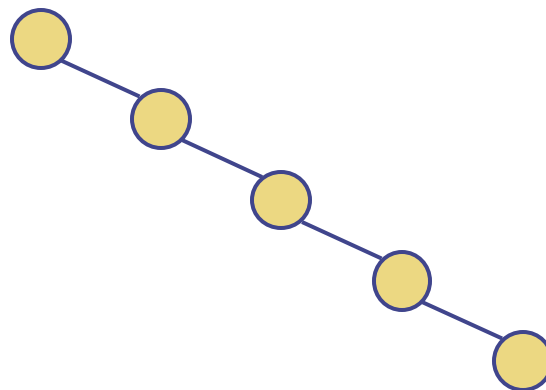
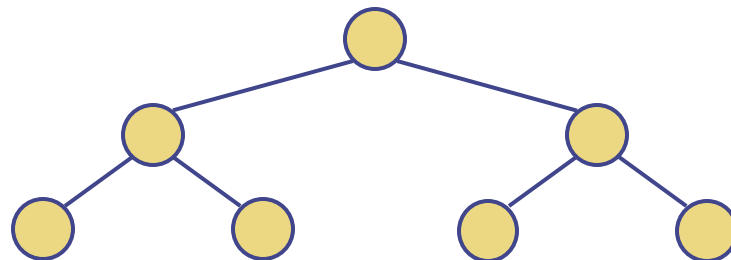
性能分析

- 分析搜索操作时间：
 - 二叉搜索算法为递归算法，每个递归调用执行常数次操作
 - 递归调用发生在从根节点向下的一条路径中的每一个节点上
 - 因此，二叉搜索的运行时间与树的高度 h 有关，为 $O(h)$



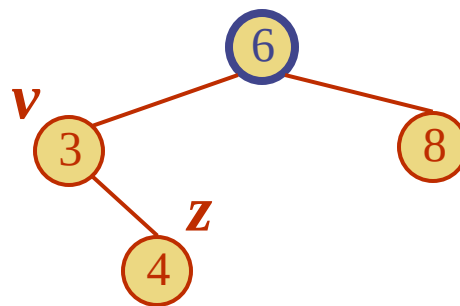
性能分析

- 插入、删除操作依赖于搜索操作，其运行时间也为 $O(h)$
- 二叉搜索树的性能很大程度上由其高度 h 决定
 - 最好情况下， h 为 $O(\log n)$ ，此时二叉搜索树为一高效数据结构，搜索操作运行时间为 $O(\log n)$
 - 最坏情况下，二叉搜索树退化为链表，搜索操作运行时间为 $O(n)$



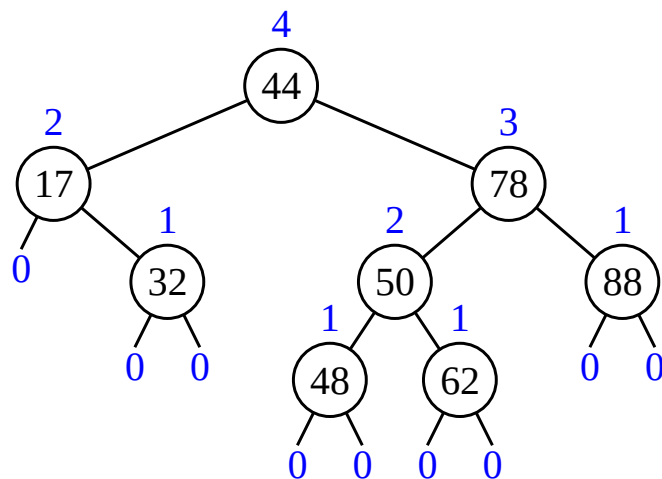


AVL树



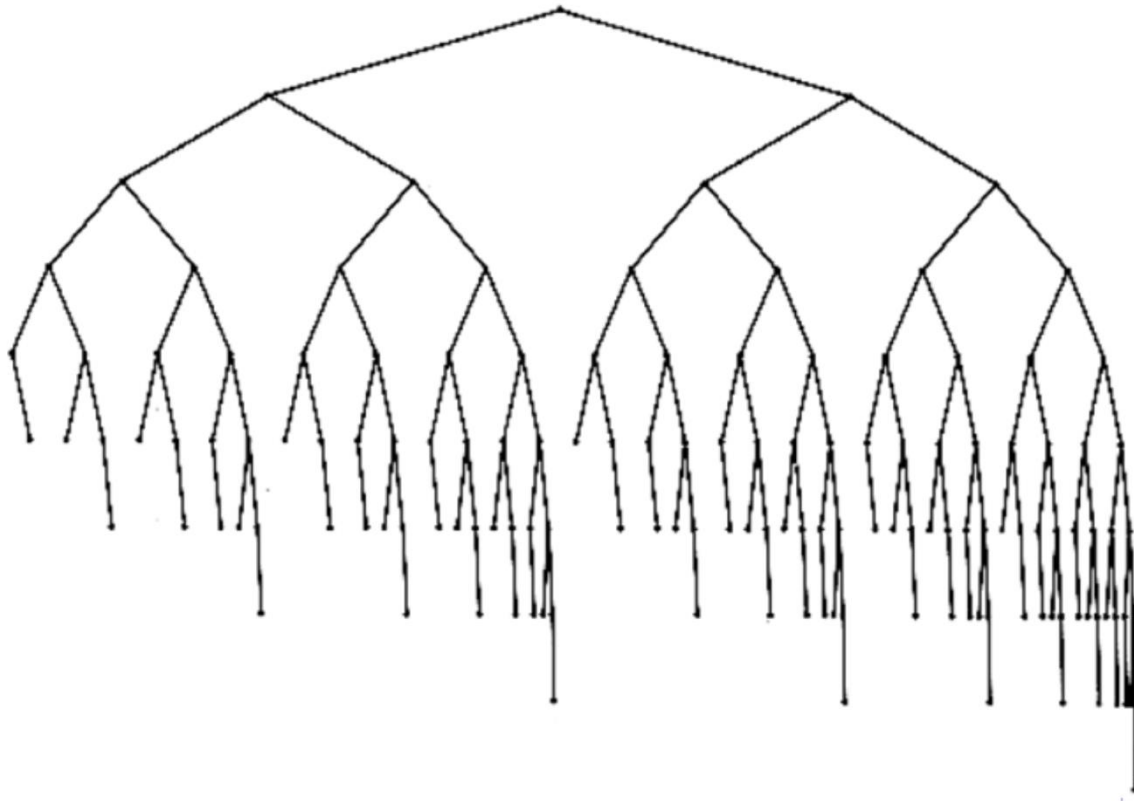
AVL树

- 我们希望构建的二叉搜索树是高效的、平衡的 (balanced) , 即其高度为 $O(\log n)$
- AVL (Adelson-Velskii and Landis) 树是一种平衡二叉搜索树
- AVL树的重要属性为以下的高度平衡属性 (Height-Balance Property) :
 - AVL树中的每个节点的孩子高度最多相差1



AVL树

- 最坏情况下的AVL树：

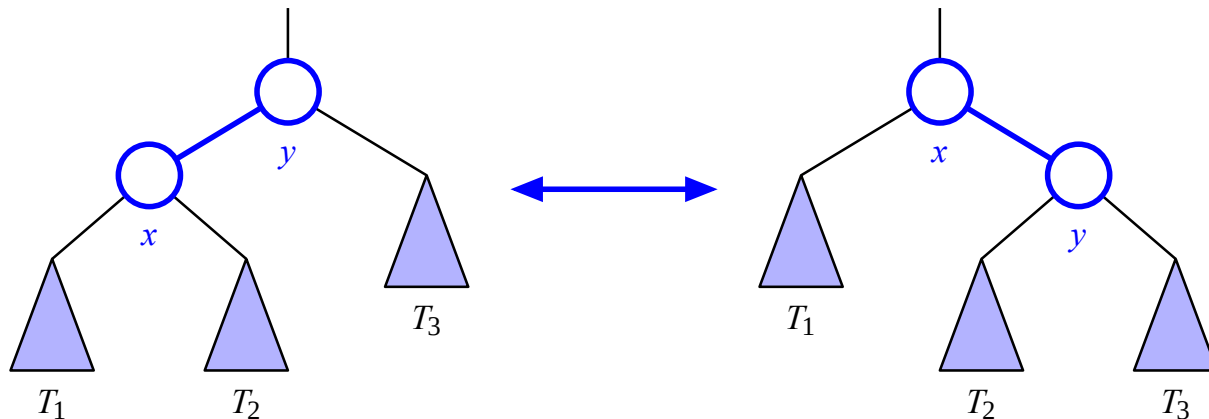


AVL树的高度

- 命题：一棵存有 n 个节点的AVL树的高度为 $O(\log n)$
- 证明：考虑相反的问题，即高度为 h 的树的最小节点数目 $n(h)$
 - 易得 $n(1) = 1$ 且 $n(2) = 2$
 - 对于 $h > 2$ 的情况，如其节点数目最少，则其两棵子树必然一棵高度为 $h - 1$ ，另一棵高度为 $h - 2$ 。考虑根节点，则可得：
$$n(h) = 1 + n(h - 1) + n(h - 2)$$
 - 如熟知斐波那契数列，则可知 $n(h)$ 为 h 的指数函数
 - $n(h) > 2n(h - 2) > 2^2n(h - 4) > \dots > 2^in(h - 2i)$ ，结合 $n(0)$ 和 $n(1)$ 的值可得 $n(h) > 2^{h/2-1}$
 - 取对数，可得 $h < 2\log(n(h)) + 2$ ，故AVL树的高度为 $O(\log n)$

AVL树的旋转与trinode重构

- AVL树的插入、删除操作类似于标准的二叉搜索树的操作，但为了保持平衡属性，需要在每次操作后进行调整
- 保持高度平衡的主要操作为**旋转 (rotation)**，及基于旋转的**trinode重构 (trinode restructuring)**
- **旋转 (zig)**：将一个孩子节点旋转到其父亲节点上方，左图旋转至右图被称为**右旋转 (right rotation)**，反之为**左旋转 (left rotation)**。下图中右旋转后可将子树 T_1 的节点深度减1，并将子树 T_3 的节点深度加1

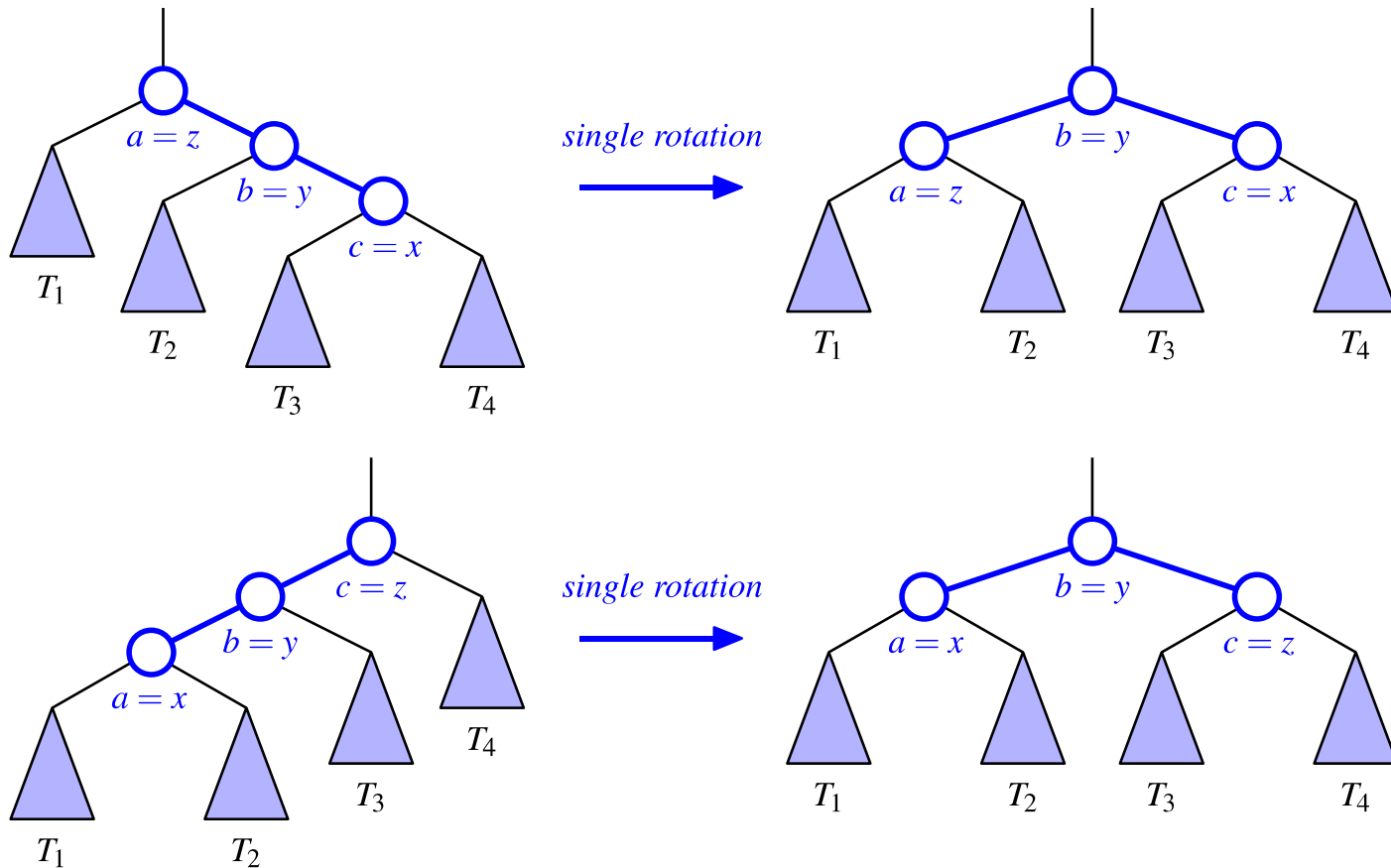


AVL树的trinode重构

- **Trinode重构 (trinode restructuring)** : 考虑对三个节点的操作, 使用1-2次旋转。假设我们考虑的位置为x, 其父节点为y, 祖父节点为z; 以z为根节点的树的中序遍历中, 这三个节点分别为a、b、c
 - **目标**: 重建以z为根的子树, **缩短到x及其子树的总路径长度**
 - **两种情况需要1次旋转, 两种情况需要2次旋转**

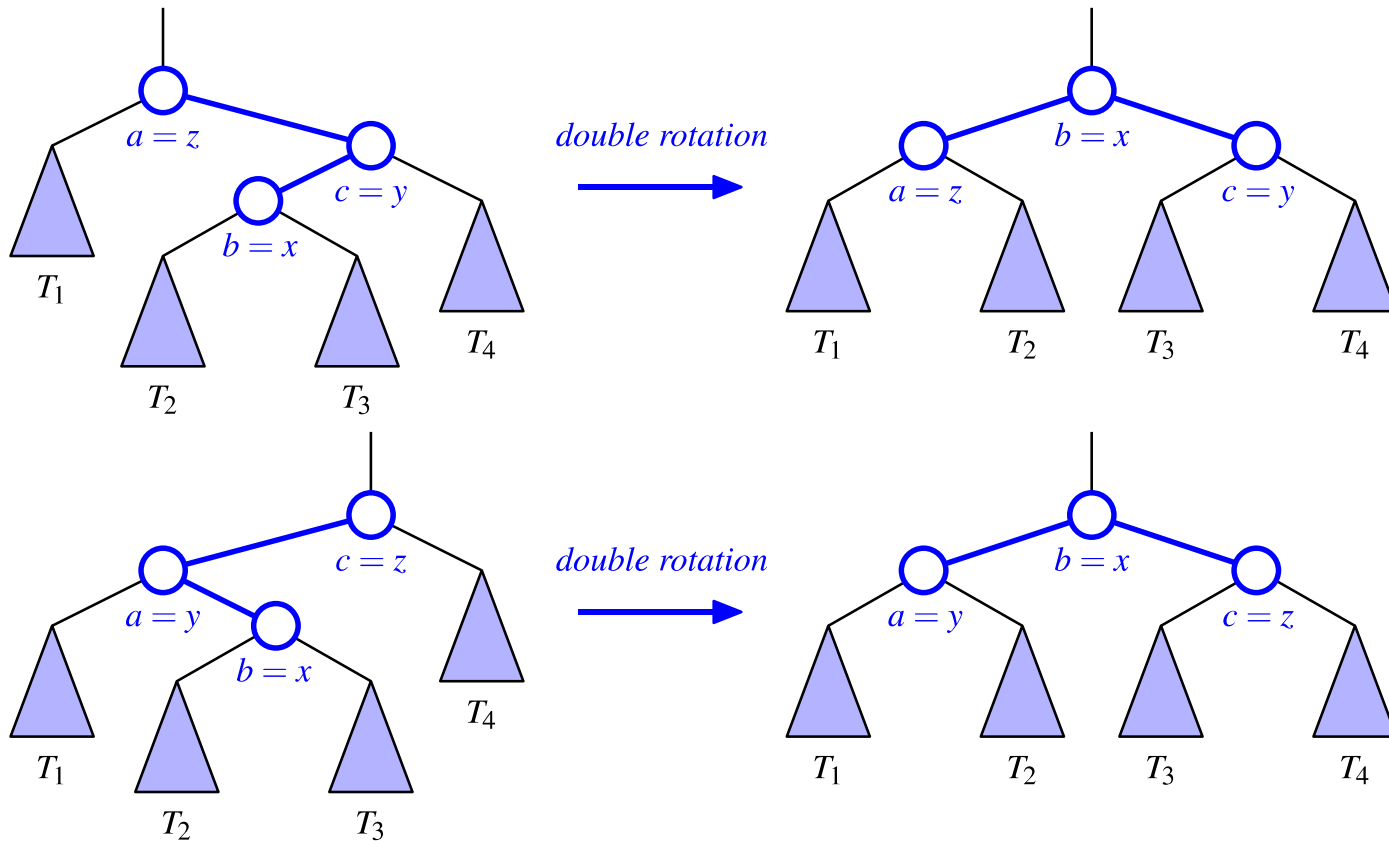
AVL树的trinode重构

- 需要1次旋转 (single rotation) 的情况 (zig-zig) :



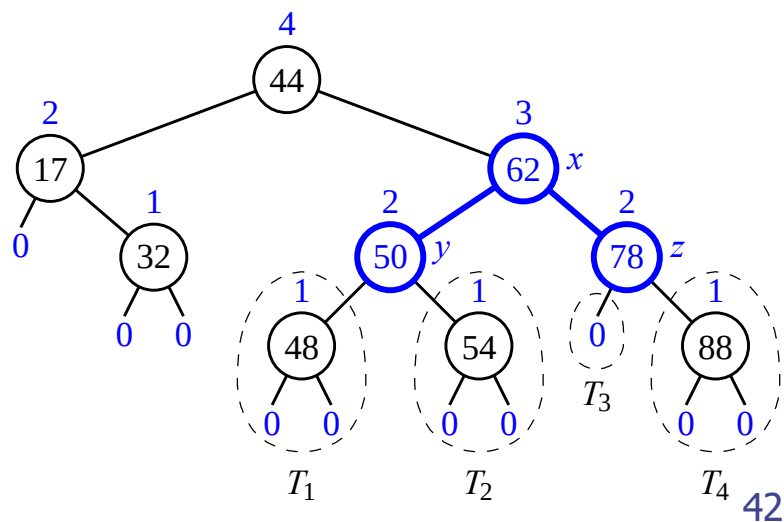
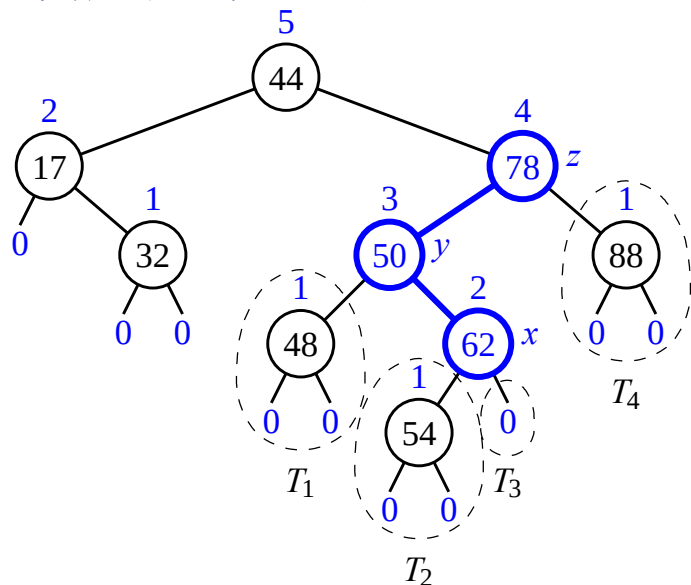
AVL树的trinode重构

- 需要2次旋转 (double rotations) 的情况 (zig-zag) :



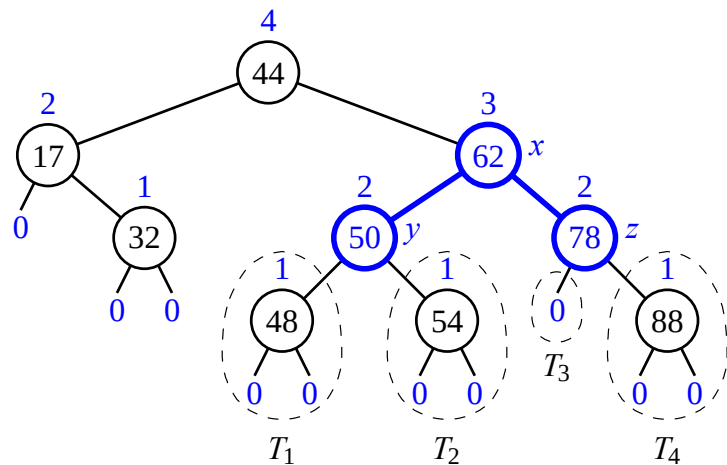
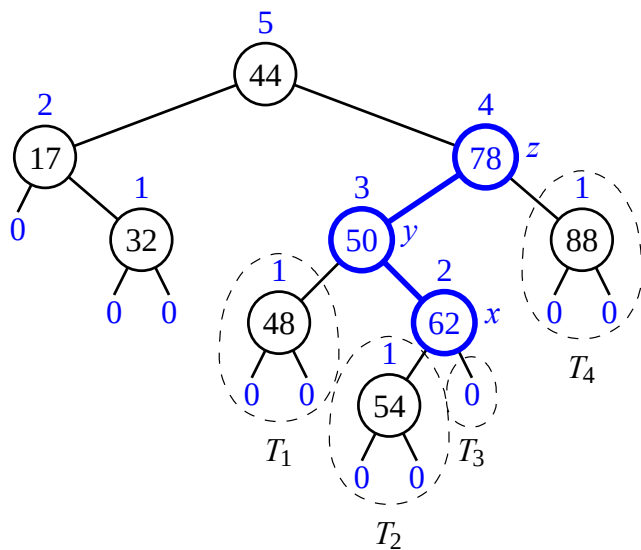
AVL树的插入

- 首先以二叉搜索树的插入方式进行插入操作
- 插入操作可能导致新节点p及其祖先处的高度平衡属性被破坏，需要通过trinode重构恢复
- 例：左图为插入54后不平衡的AVL树，右图为重构后恢复高度平衡属性的AVL树



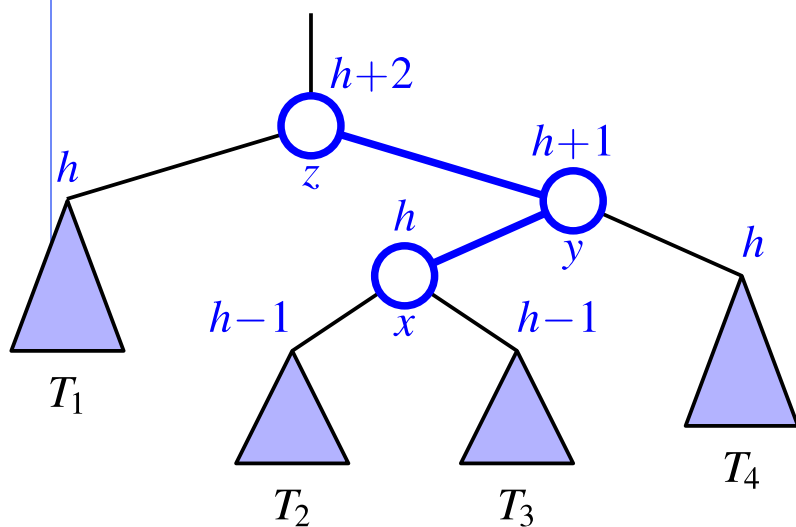
AVL树的插入

- 我们通过一个简单的“查找-修复”策略来恢复AVL树中节点的平衡
- z 表示从新节点 p 到根的路径上遇到的第一个不平衡位置
- y 表示 z 具有更高高度的孩子（ y 必为 p 的祖先）
- x 表示 y 具有更高高度的孩子（ x 必为 p 的祖先或 p 自身）
- 在 x 处进行trinode重构，即可使以 z 为根的子树重新平衡

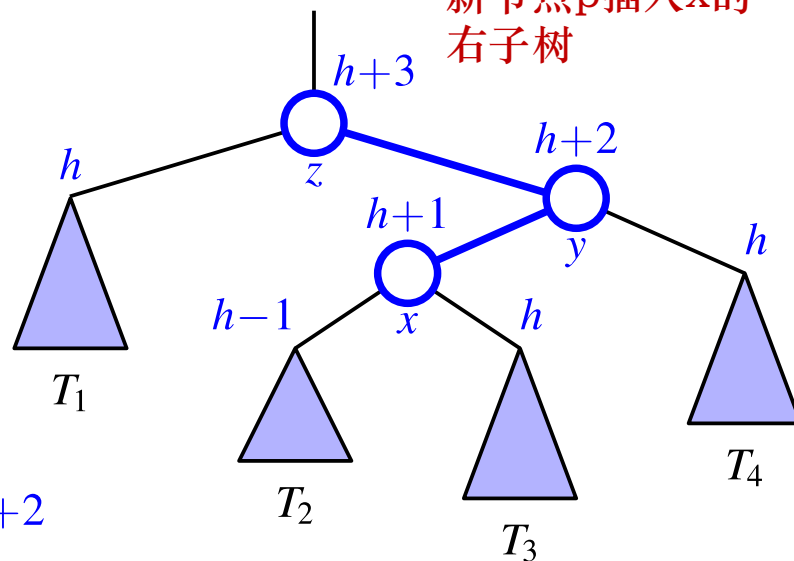


AVL树的插入

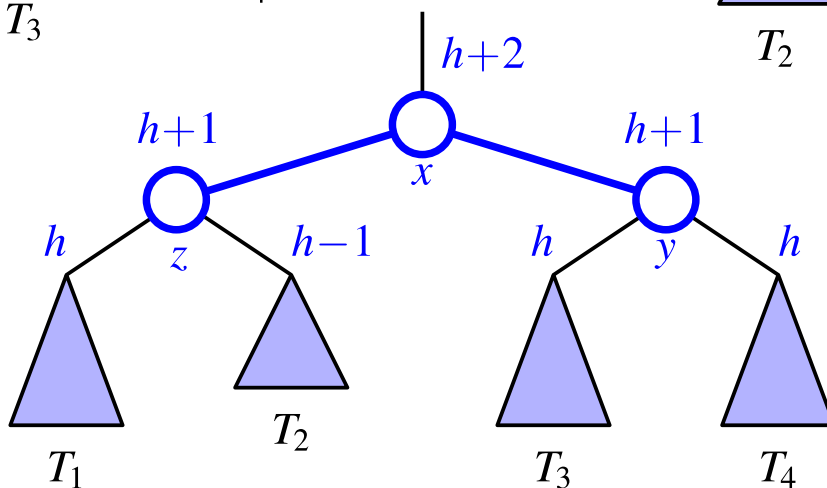
· 例:



新节点p插入x的
右子树

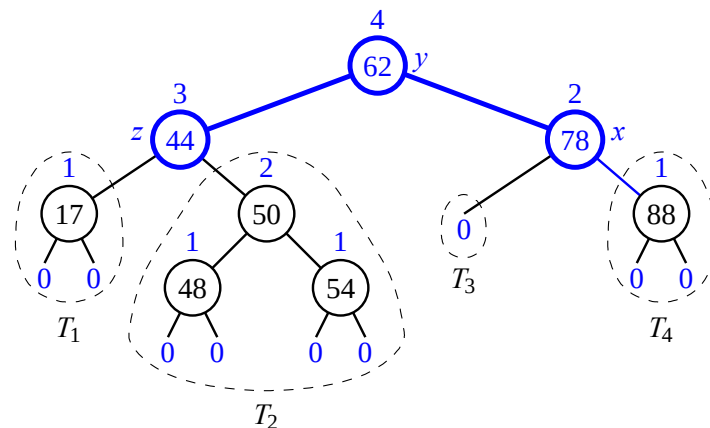
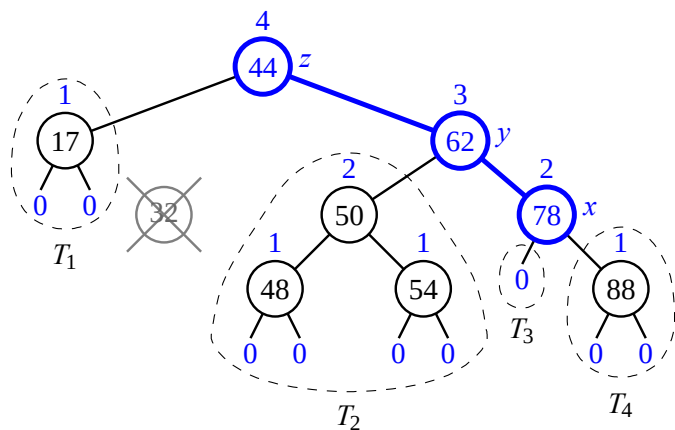


trinode重构后



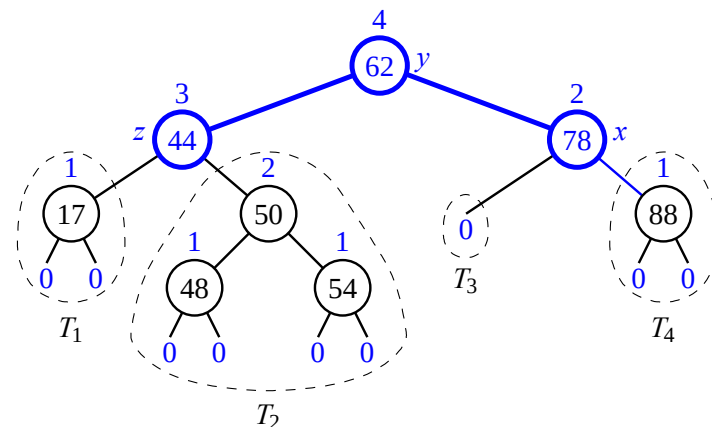
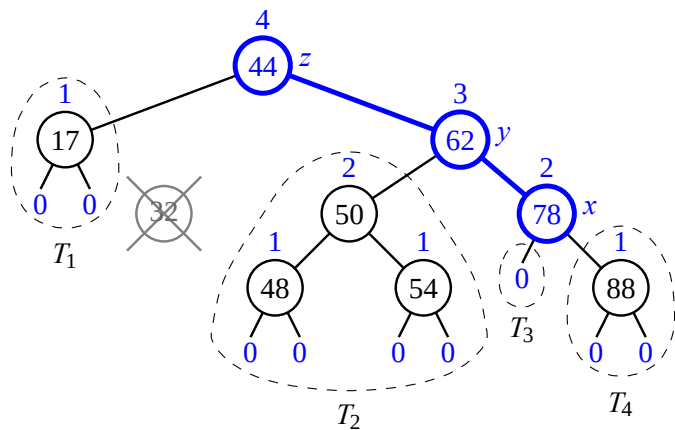
AVL树的删除

- 首先以二叉搜索树的删除方式进行删除操作
- 被删除节点的父节点p到根节点的路径上可能存在一个不平衡的节点，需要通过trinode重构恢复高度平衡
- 例：左图为删除32后不平衡的AVL树，右图为重构后恢复高度平衡属性的AVL树



AVL树的删除

- z 表示从新节点 p 到根的路径上遇到的第一个不平衡位置
- y 表示 z 具有更高高度的孩子（ y 必不为 p 的祖先）
- x 的定义方式如下：
 - 如 y 的两个孩子高度不同，则令 x 为 y 较高的孩子
 - 否则，令 x 为与 y 在同一侧的孩子，即使得trinode重构只需一次旋转
- 在 x 处进行trinode重构，即可使以 z 为根的子树重新平衡

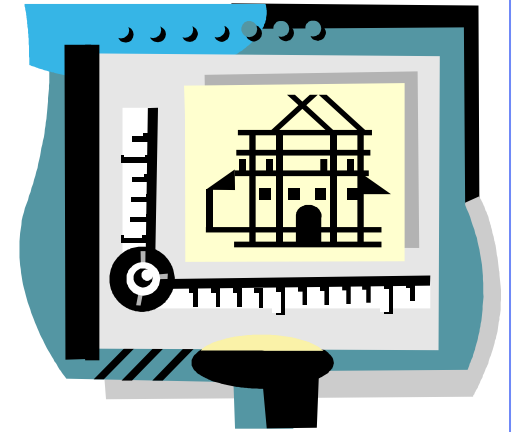


AVL树的删除

- 在x处的trinode重构使得原来以z为根的子树高度减少1，这可能会导致这棵子树根节点的祖先变得不平衡
- 进行一次trinode重构后，需要沿着通向根节点的路径继续检查是否有不平衡的位置
- 由于AVL树为平衡二叉树，故在 $O(\log n)$ 时间内可完成所有的trinode重构，恢复AVL树的高度平衡属性

AVL树的性能分析

- 一次trinode重构运行时间为 $O(1)$
- 搜索的运行时间为 $O(\log n)$
 - AVL树的高度为 $O(\log n)$ ；搜索不需要trinode重构
- 插入的运行时间为 $O(\log n)$
 - 找到插入位置需要的时间为 $O(\log n)$
 - 进行一次trinode重构需要的时间为 $O(1)$ ；更新节点高度信息需要的时间为 $O(\log n)$
- 删除的运行时间为 $O(\log n)$
 - 找到删除位置需要的时间为 $O(\log n)$
 - 进行trinode重构需要的时间为 $O(\log n)$ ；更新节点高度信息需要的时间为 $O(\log n)$



二叉搜索树的实现（续）

```
def _relink(self, parent, child, make_left_child):
    """Relink parent node with child node (we allow child to be None)."""
    if make_left_child:
        parent._left = child
    else:
        parent._right = child
    if child is not None:
        child._parent = parent

def _rotate(self, p):
    """Rotate Node p above its parent."""
    x = p
    y = x._parent
    z = y._parent
    if z is None:
        self._root = x
        x._parent = None
    else:
        self._relink(z, x, y == z._left)
    # now rotate x and y, including transfer of middle subtree
    if x == y._left:
        self._relink(y, x._right, True)
        self._relink(x, y, False)
    else:
        self._relink(y, x._left, False)
        self._relink(x, y, True)
```

we assume this exists
grandparent (possibly None)
x becomes root
x becomes a direct child of z
x._right becomes left child of y
y becomes right child of x
x._left becomes right child of y
y becomes left child of x

二叉搜索树的实现（续）

```
def _restructure(self, x):  
    """Perform trinode restructure of Node x with parent/grandparent."""  
    y = self.parent(x)  
    z = self.parent(y)  
    if (x == self.right(y)) == (y == self.right(z)): # matching alignments  
        self._rotate(y)                               # single rotation (of y)  
        return y                                       # y is new subtree root  
    else:                                              # opposite alignments  
        self._rotate(x)                               # double rotation (of x)  
        self._rotate(x)  
        return x                                     # x is new subtree root
```


AVL树的实现

```
class AVLTreeMap(TreeMap):
    """Sorted map implementation using an AVL tree."""

    #----- nested Node class -----
    class Node(TreeMap.Node):
        """Node class for AVL maintains height value for balancing.

        We use convention that a "None" child has height 0, thus a leaf has height 1.
        """
        __slots__ = '_height'          # additional data member to store height

        def __init__(self, element, parent=None, left=None, right=None):
            super().__init__(element, parent, left, right)
            self._height = 0           # will be recomputed during balancing

        def left_height(self):
            return self._left._height if self._left is not None else 0

        def right_height(self):
            return self._right._height if self._right is not None else 0
```

AVL树的实现

```
#----- nodal-based utility methods -----  
def _recompute_height(self, p):  
    p._height = 1 + max(p.left_height(), p.right_height())  
  
def _isbalanced(self, p):  
    return abs(p.left_height() - p.right_height()) <= 1  
  
def _tall_child(self, p, favorleft=False): # parameter controls tiebreaker  
    if p.left_height() + (1 if favorleft else 0) > p.right_height():  
        return self.left(p)  
    else:  
        return self.right(p)  
  
def _tall_grandchild(self, p):  
    child = self._tall_child(p)  
    # if child is on left, favor left grandchild; else favor right grandchild  
    alignment = (child == self.left(p))  
    return self._tall_child(child, alignment)
```

AVL树的实现

```
def _rebalance(self, p):
    while p is not None:
        old_height = p._height                # trivially 0 if new node
        if not self._isbalanced(p):           # imbalance detected!
            # perform trinode restructuring, setting p to resulting root,
            # and recompute new local heights after the restructuring
            p = self._restructure(self._tall_grandchild(p))
            self._recompute_height(self.left(p))
            self._recompute_height(self.right(p))
        self._recompute_height(p)              # adjust for recent changes
        if p._height == old_height:           # has height changed?
            p = None                           # no further changes needed
        else:
            p = self.parent(p)                 # repeat with parent

#----- override balancing hooks -----
def _rebalance_insert(self, p):
    self._rebalance(p)

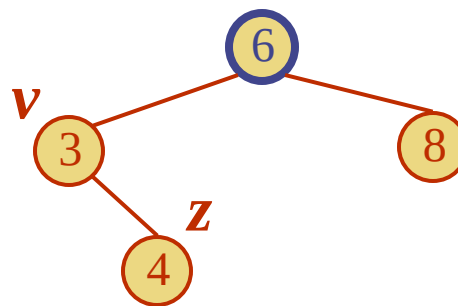
def _rebalance_delete(self, p):
    self._rebalance(p)
```

ANY
QUESTIONS
?

www.rekrutlin.com

ReKrutlin.com

伸展树



伸展树



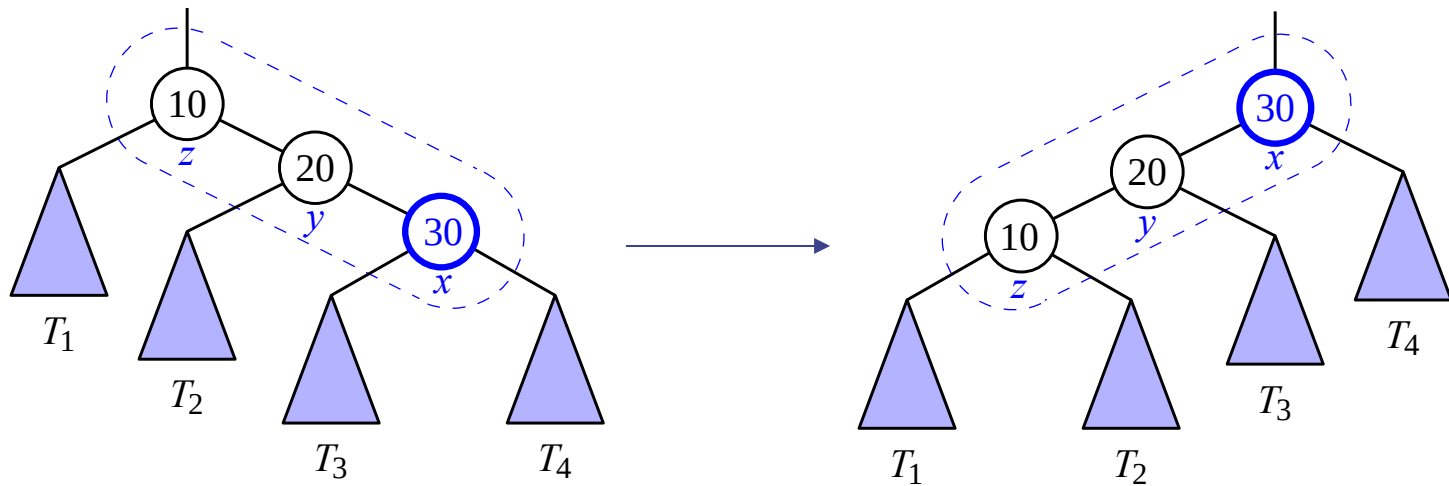
- 伸展树 (splay tree) 与本课中涉及的其他平衡搜索树有显著的不同
 - 伸展树的高度没有严格的对数上界
 - 伸展树的节点处无需存储额外的高度、平衡或其他辅助信息
- 在伸展树上进行搜索、插入、删除操作后，均需在到达的最后一个节点处进行伸展 (splaying) 操作
 - 直观上，伸展操作使得被频繁操作的节点更靠近树根

伸展

- 对某一节点 x 的伸展操作即通过一系列重构将其移动到根节点的操作
- 根据节点 x 是否有祖父节点及其与父节点及祖父节点（如存在）间的位置关系，可以采用zig-zig、zig-zag或zig重构
 - 有祖父节点时执行zig-zig或zig-zag
 - 没有祖父节点时执行zig

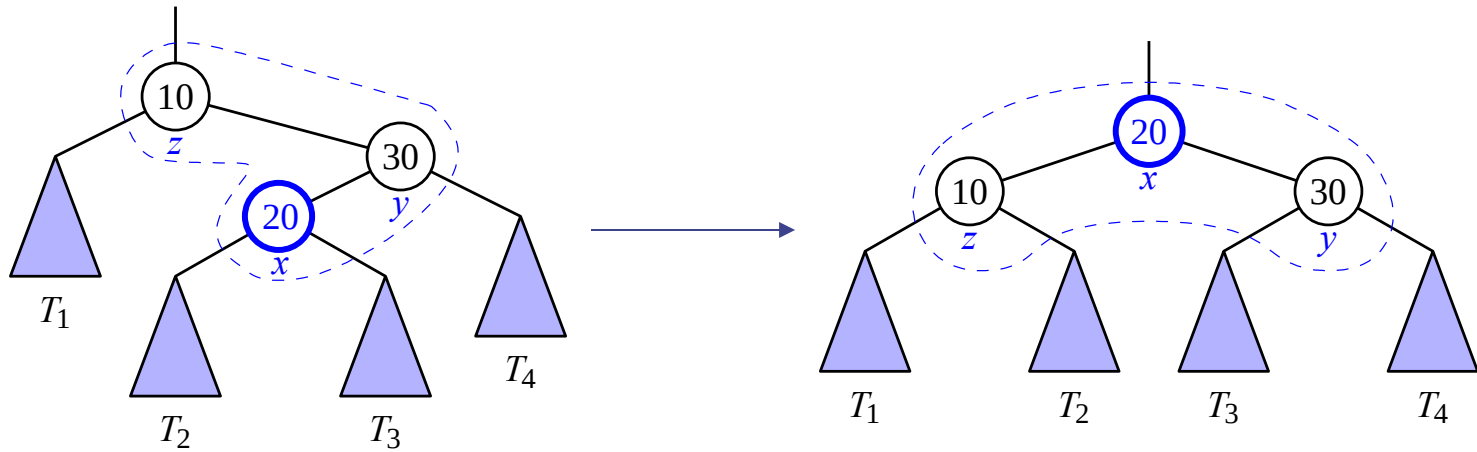
伸展

- zig-zig型:



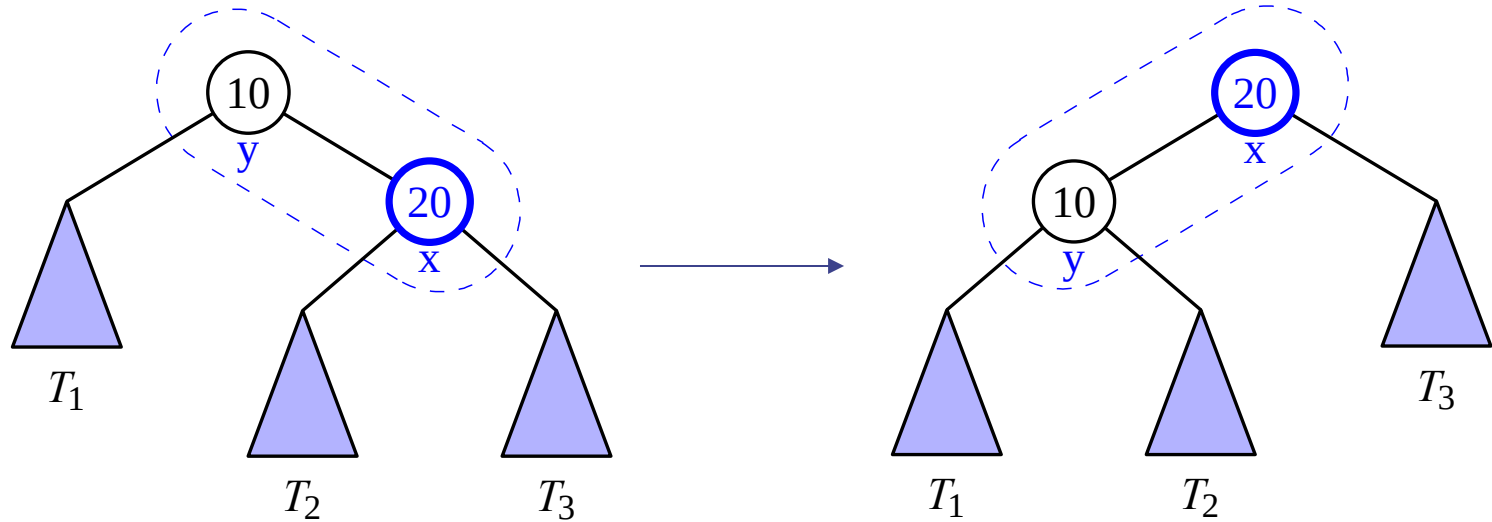
伸展

- zig-zag型:



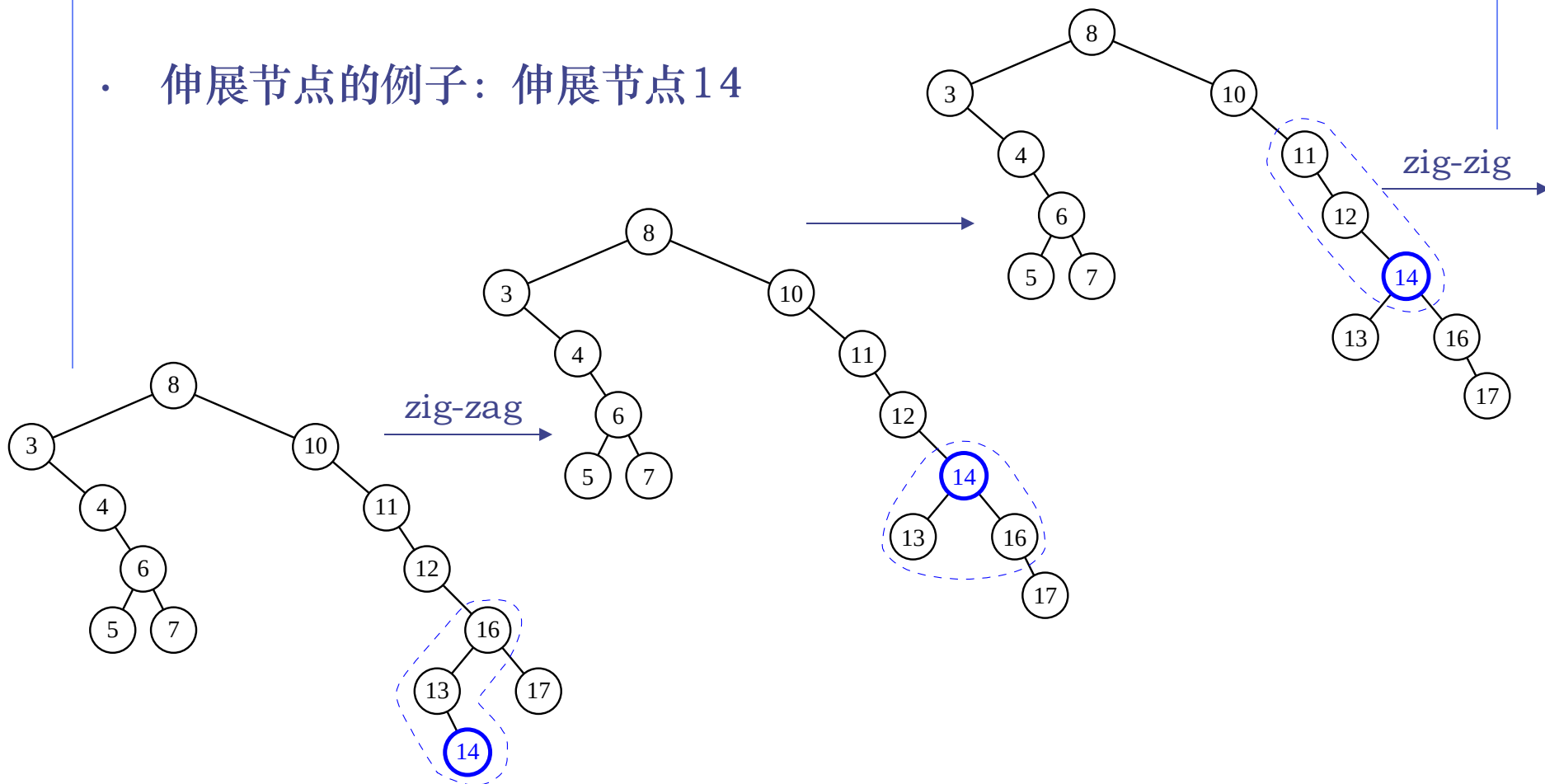
伸展

- zig型:



伸展

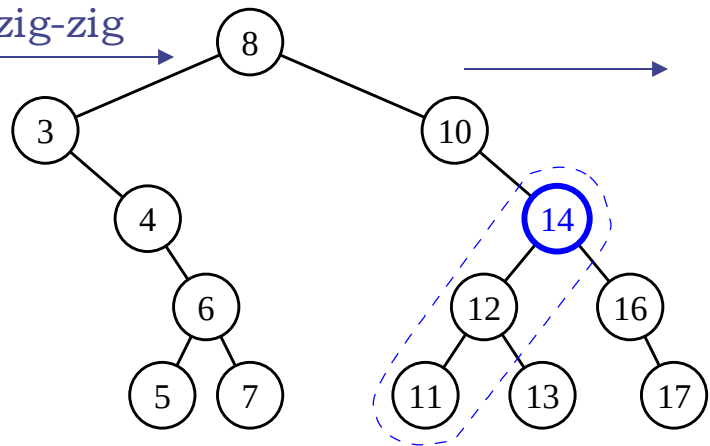
- 伸展节点的例子：伸展节点14



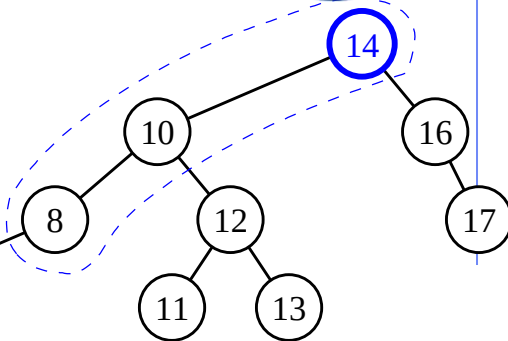
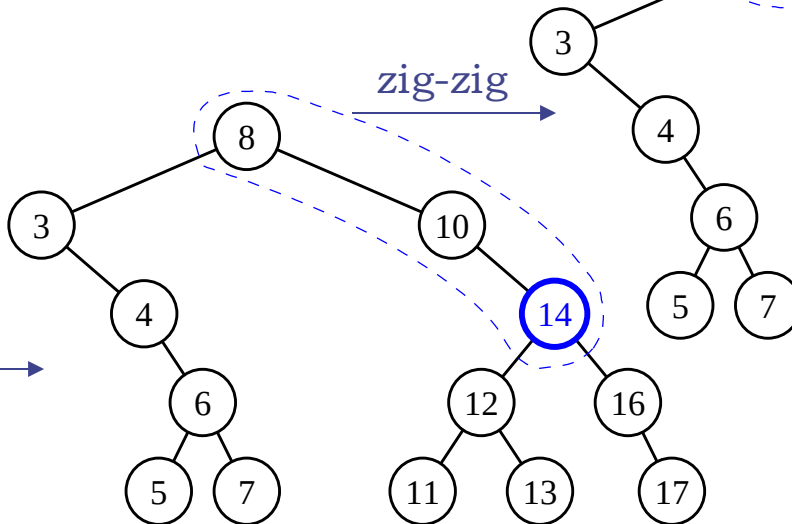
伸展

- 伸展节点的例子：伸展节点14

zig-zig



zig-zig

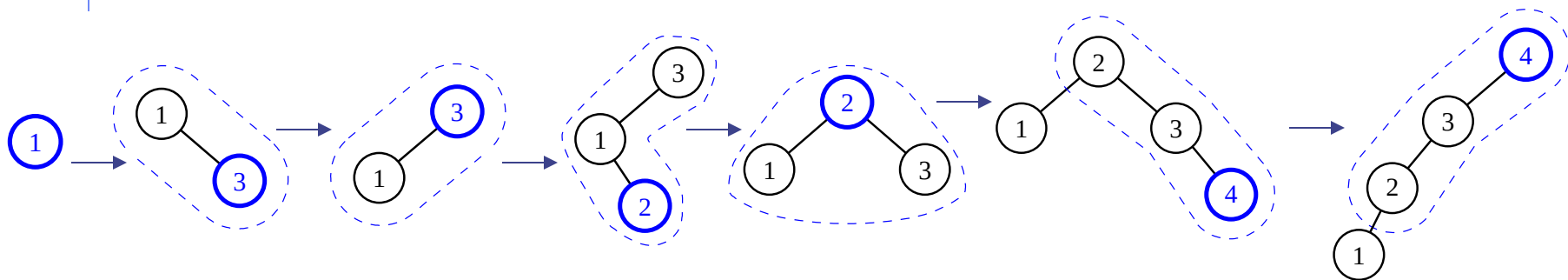


伸展树的搜索

- 搜索键 k 时，如果发现键 k 在位置 p ，则伸展 p ；否则，在搜索失败的位置伸展叶子节点
 - 前两页课件展示的即为搜索键14成功或键15失败时的伸展情况

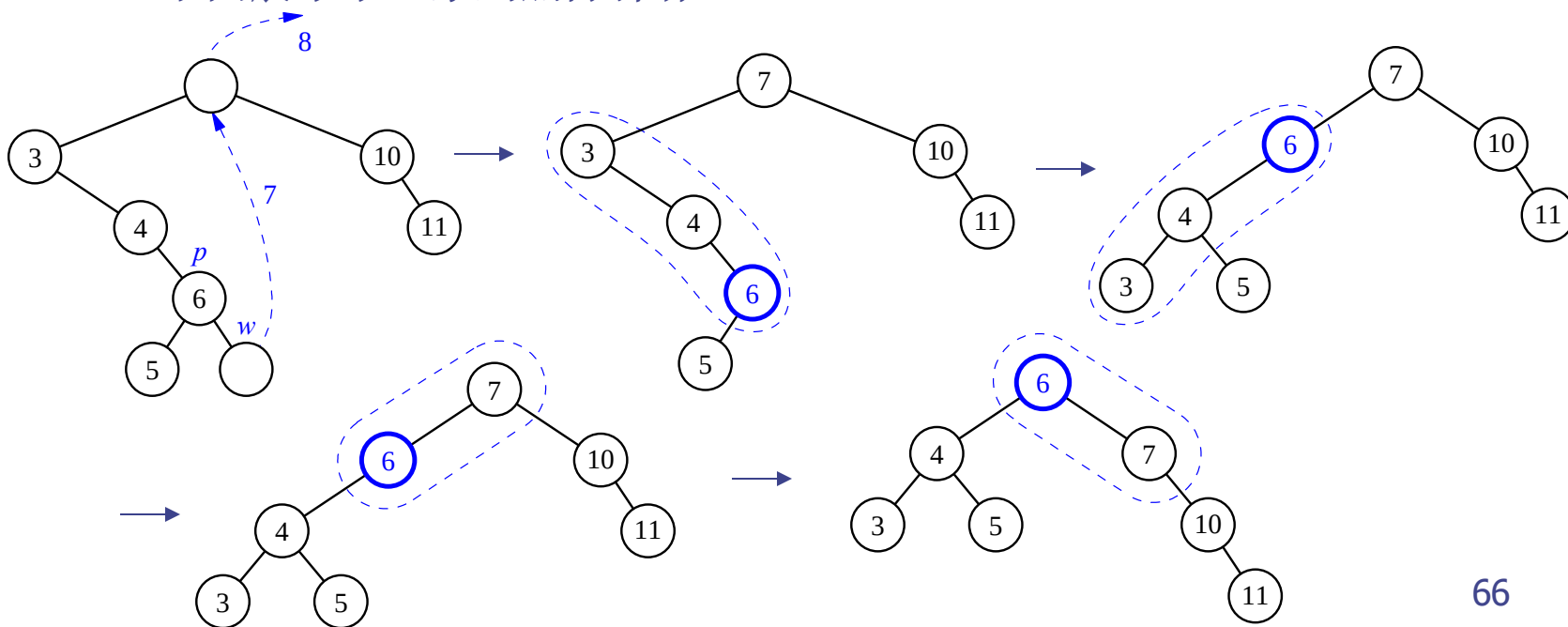
伸展树的插入

- 插入键 k 时，在新插入的叶子节点 p 处进行伸展
 - 前两页课件展示的为插入键14时的伸展情况
 - 下图展示了一系列插入操作



伸展树的删除

- 删除键 k 时，在被删除节点的父节点 p 处进行伸展
 - 被删除节点可能包含键 k ，也有可能是一个用来替代被删除节点的节点
 - 下图展示了一系列删除操作



伸展树的性能



- 伸展操作的摊销运行时间为 $O(\log n)$ ，故搜索、插入、删除的摊销运行时间也为 $O(\log n)$
- 优势：
 - 无需在节点处存储额外信息，且操作简单
 - 某些情况下运行时间远小于 $O(\log n)$ ，如当操作经常被操作的节点时

伸展树的实现

```
class SplayTreeMap(TreeMap):
    """Sorted map implementation using a splay tree."""

    #----- splay operation -----
    def _splay(self, p):
        while p != self.root():
            parent = self.parent(p)
            grand = self.parent(parent)
            if grand is None:
                # zig case
                self._rotate(p)
            elif (parent == self.left(grand)) == (p == self.left(parent)):
                # zig-zig case
                self._rotate(parent)          # move PARENT up
                self._rotate(p)               # then move p up
            else:
                # zig-zag case
                self._rotate(p)               # move p up
                self._rotate(p)               # move p up again
```



Enjoy